Teaching a Neural Network to Follow a Chaotic System

Key findings in eight graphs, explained without the math

This is a companion to the full report (Lorenz_PINN_Report.pdf, Sections 16 and 17). It walks through the eight graphs that tell the story, one at a time, in plain language. Each graph gets three parts: what is drawn on it, what it shows, and why it matters.

Before the graphs: the ideas you need

The system we are trying to predict

The **Lorenz system** is a set of three simple equations, first written down in 1963 as a toy model of air moving in the atmosphere. It tracks three numbers over time, called **x**, **y** and **z**. You can picture them as the position of a point moving through 3D space. The equations say, at every instant, which direction the point moves and how fast.

Plotted in 3D, the point traces the famous "butterfly" shape. It loops around one wing for a while, then jumps over and loops around the other wing, and nobody can tell in advance when it will jump. On the path used here, one loop takes about **0.62 time units**. The first jump to the other wing is at about t = 0.5, and later jumps happen around t = 13.6, 14.3, 17.2, 18.1 and 18.9.

The system is **chaotic**. This has a specific meaning: two starting points that differ by a tiny amount drift apart faster and faster over time. A difference of one in a million can grow to the size of the whole butterfly. So even a perfect method has some limit on how far ahead it can stay accurate, because tiny errors always creep in and chaos grows them.

What we are using to predict it

A **PINN** (physics-informed neural network) is a neural network that takes in a time t and gives back a guess for x, y and z at that time. It is trained in two ways at once:

- **Data:** at some times we know the true x, y and z (computed very accurately with a standard step-by-step solver). The network is penalised when its guess is far from these values.
- **Physics:** at many other times we check whether the network's curve obeys the Lorenz equations, i.e. whether it is moving in the direction the equations say it should. The network is penalised when it doesn't. This needs no data at all, only the equations.
- The number that measures how badly it is doing is called the **loss**. Training means adjusting the network, over thousands of small steps, to push the loss down. A loss near 0 means the network fits both the data and the physics.

Every network here is the same size (4 layers of 64 units) and starts at the right point: at t = 0 it is forced to give x = y = z = 1. Training uses two standard methods, one after the other: Adam (10,000 steps) and then L-BFGS (up to 5,000 steps), which polishes the result.

One network versus time-marching

- **One network:** a single network is trained to cover the whole stretch of time, from 0 to T, all at once.
- **Time-marching:** the time is chopped into short **windows** (for example 1 unit each: 0 to 1, 1 to 2, and so on). Each window gets its own fresh network. The first network starts at the true starting point. Each later network starts exactly where the previous network's prediction ended. That end point is the only thing passed from one window to the next, which is why we call it the **hand-off**.
- **Warm start:** an option in time-marching where each new window's network begins training as a copy of the previous window's trained network, instead of from random settings ("fresh weights").

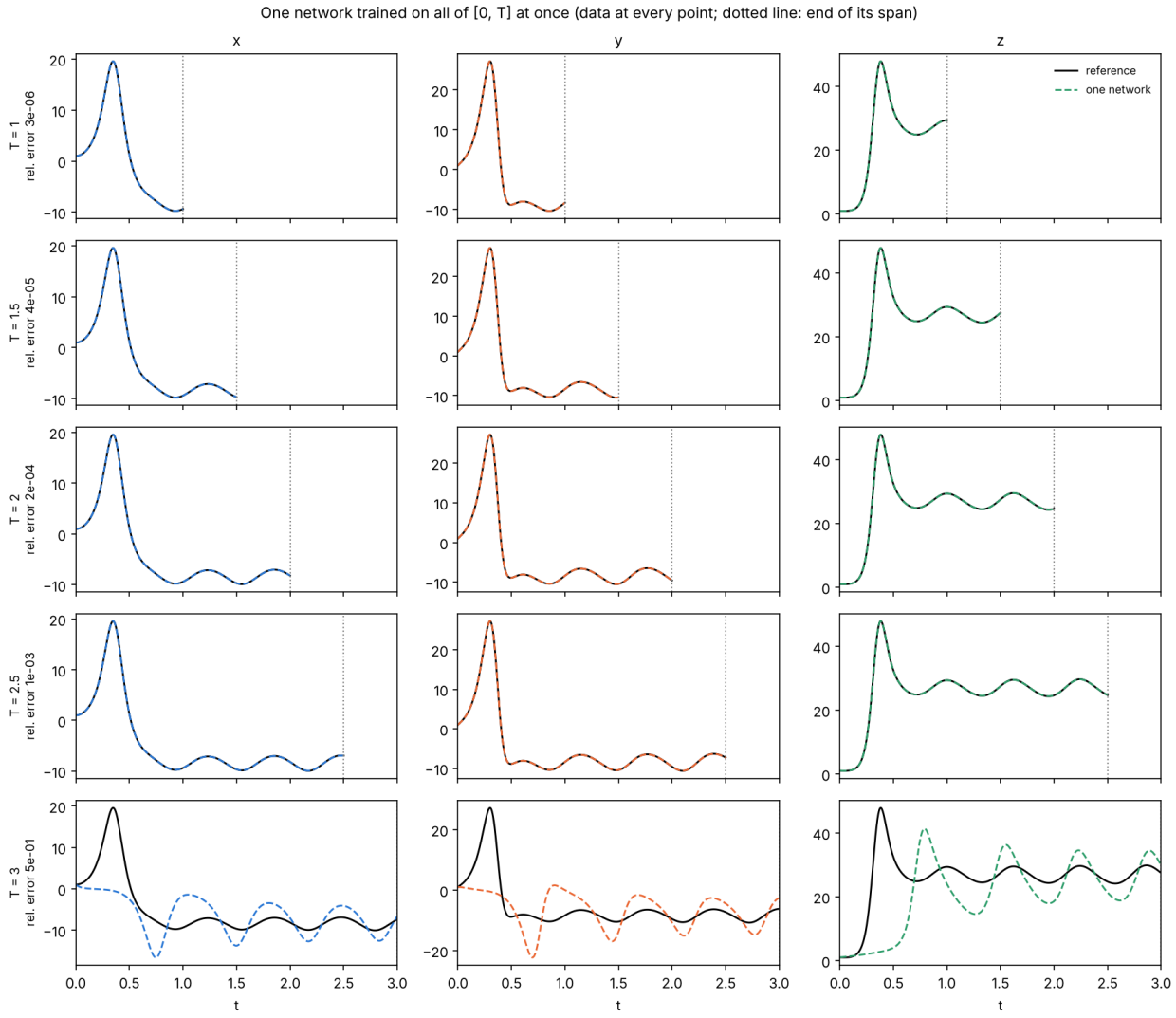
The idea is that neighbouring windows look alike, so a copy should be a good head start. As the graphs show, that idea turned out to be wrong here.

How to read the error graphs

- **Error** on these graphs means the straight-line distance in 3D between where the network says the point is and where it really is. The butterfly is about 40 to 50 units across, so an error of 1 is a noticeable miss, and an error of 10 means the network is somewhere else entirely.
- Most error graphs use a **log scale** on the vertical axis. Each gridline is 10 times the one below it: 10^{-4} is 0.0001, 10^{-2} is 0.01, 10^{0} is 1. Equal steps up the axis mean multiplying the error by 10, not adding to it. That lets one graph show both tiny and huge errors.
- The **dashed line at 10^{-4}** (0.0001) is the accuracy target used throughout the project. Below it, the prediction is essentially on top of the true path.
- "Relative error" means the error divided by the size of the true values, so 10^{-6} means the network is off by about one part in a million.

Graph 1: One network as the time span grows

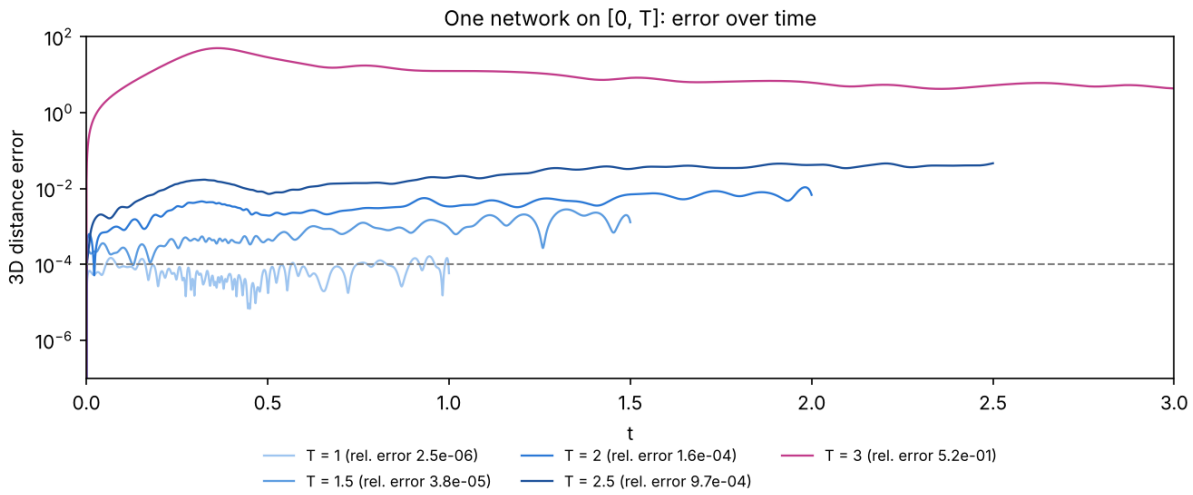
One network on [0, T] for T = 1, 1.5, 2, 2.5 and 3.



Graph 1a. x, y and z for one network trained on all of [0, T], for each T (one row per T). Solid black: the true path. Dashed: the network. Dotted line: the end of the span.

What you are looking at

Five separate experiments, one per row. In each, a single network is trained on the whole span from 0 to T at once, with true data at every point and the physics checked everywhere. Nothing changes between rows except T, the length of time the network has to cover. The three columns are x, y and z. The label on the left of each row gives the overall relative error for that run.



Graph 1b. The same five runs as error over time on a log scale. The dashed line is the 10^{-4} target.

What it shows

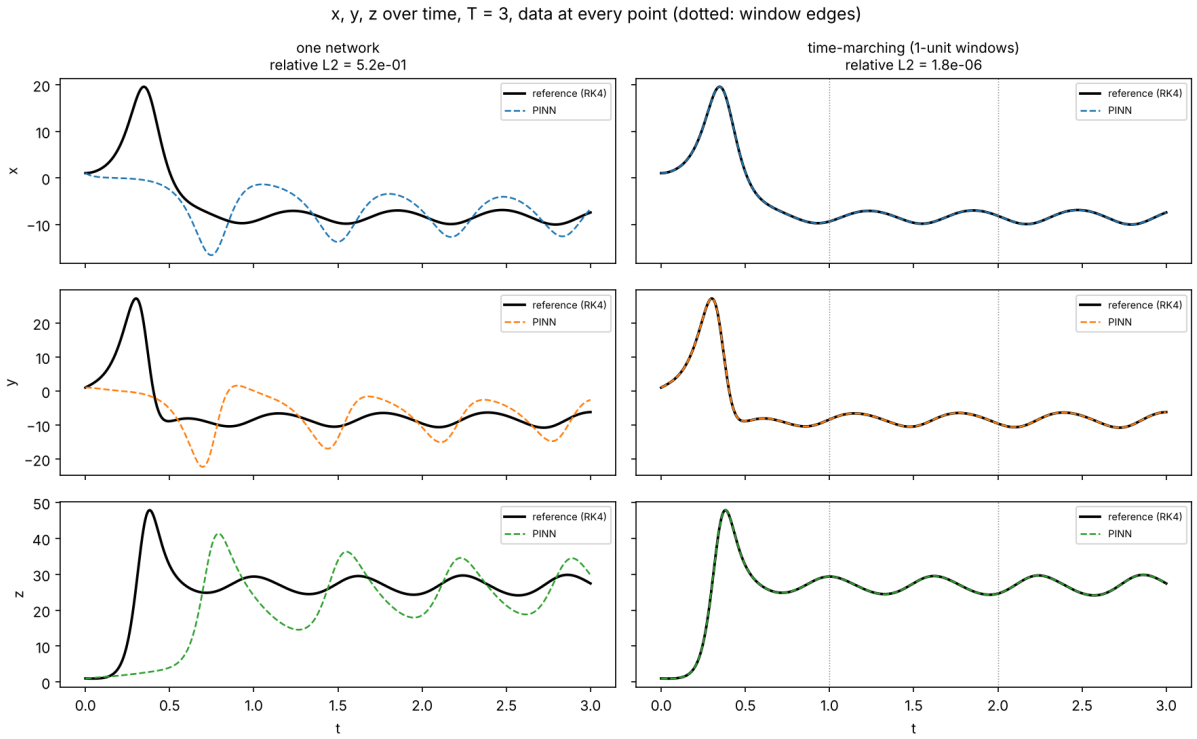
- **T = 1 to 2.5:** the dashed line sits on the true path in every row. The network catches the big early swing (x up to 20, z up to 48 near t = 0.3) and then follows the calm loops that come after it.
- **But accuracy drops steadily as T grows,** even while the curves still look right. Overall relative error: 2.5×10^{-6} at T = 1, 3.8×10^{-5} at 1.5, 1.6×10^{-4} at 2, 9.7×10^{-4} at 2.5. That is roughly 4 to 15 times worse for every extra half unit. In Graph 1b, only T = 1 stays at the 10^{-4} target; the others sit above it, and their worst points reach 0.003 at T = 1.5, 0.01 at T = 2 and 0.05 at T = 2.5.
- **T = 3: complete failure.** The bottom row is wrong from the start. The network misses the early swing entirely, then swings too hard and at the wrong times. Relative error jumps to 0.52 (about 50% off), and in Graph 1b the error leaps by about 500 times compared with T = 2.5.
- Remember that every run had data at every point. Lack of information is not the reason.

Key takeaway

A single network has a limited budget. Each extra half unit of time makes it noticeably less accurate, and somewhere between T = 2.5 and 3 it stops finding the path at all. Up to that point the decline is gradual; at that point it is sudden. The next two graphs compare the failed T = 3 run with time-marching on the same span.

Graph 2: The fix, at T = 3

One network fails at T = 3; time-marching does not.



Graph 2. x, y and z from t = 0 to 3. Left: one network. Right: time-marching with 1-unit windows. Solid: the true path. Dashed: the network's prediction.

What you are looking at

Six small plots, one row for each of x, y and z. The left column is a single network trained on the whole stretch from 0 to 3. The right column is time-marching, with three networks covering 0 to 1, 1 to 2 and 2 to 3. The solid line is the true path. The dashed line is what the network predicts. Where the two lines sit on top of each other, the prediction is right.

What it shows

- **Both methods were given true data at every point** from 0 to 3, so neither was short of information. The only difference is one network versus three.
- **Right column:** the dashed line sits exactly on the solid line everywhere. Time-marching gets the whole path right, including the big early swing where x shoots up to about 20 near t = 0.3.
- **Left column:** the single network misses that early swing completely. Where the true x rises to 20, the prediction stays near 0. After that it does wiggle, but its swings are too big and land at the wrong times, so it is never on the true path. It is wrong *from the very beginning*, not just at the end.
- This is important: if chaos were the problem, the network would start out right and drift away later. Here it is wrong from t = 0, even with data everywhere, which means it simply never learned the path.

Key takeaway

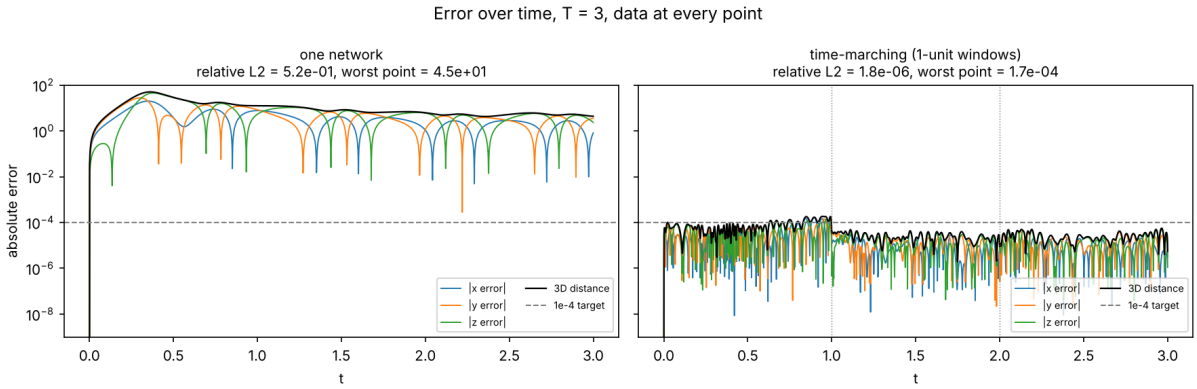
Asking one network to cover 3 time units is asking too much of it. It settles on a wrong answer that misses the sharp early swing, and once it has missed that, nothing in training pulls it back. Splitting the same time into three 1-unit pieces makes each piece easy enough to learn.

Why exactly T = 3?

As Graph 1 showed, up to T = 2.5 one network stayed on the path, though less accurately each time. At 3 it failed outright. This is a sharp tipping point, not a gradual slide. Looking at the training logs: at T = 2.5 the network finds the early spike after about 5,000 training steps, and from then on it improves quickly. At T = 3 it never finds the spike, and its loss stays stuck near 1 for the whole run. Once training misses that feature, there is nothing to pull it back, so a little more time is the difference between success and complete failure.

Graph 3: How big the gap is

The same T = 3 comparison, measured as error.



Graph 3. Error over time at T = 3 on a log scale, for x, y and z. Left: one network. Right: time-marching. The dashed line is the 10^{-4} target.

What you are looking at

The same two methods as Graph 2, but now each line is the size of the error over time, for x, y and z separately. The vertical axis is a log scale, so each gridline up means 10 times worse.

What it shows

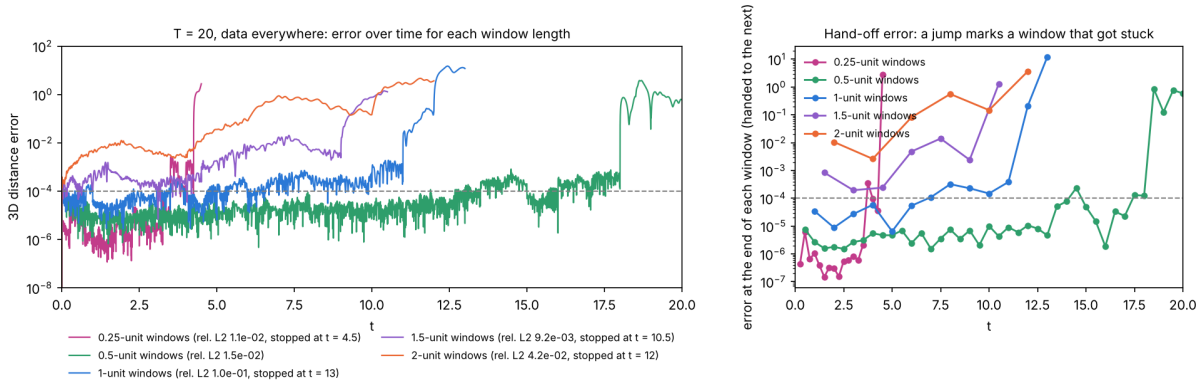
- **One network (left):** the error is around 1 to 10 for almost the whole time, and as large as about 50 at its worst. Its typical 3D miss is about 7 units on a shape about 45 units across. Overall it is off by about 50% (relative error 0.52).
- **Time-marching (right):** the error stays around 10^{-5} to 10^{-4} (0.00001 to 0.0001) the whole way, at or below the target line. Its worst point is about 0.0002. Overall it is off by about two parts in a million (relative error 1.8×10^{-6}).
- That is a gap of about **five orders of magnitude**: time-marching is roughly 100,000 times more accurate on the same problem with the same network size.
- The time-marching error does not grow steadily across the three windows. That tells us the hand-offs between windows are clean: each network starts almost exactly on the true path.

Key takeaway

The failure of one network is a **training** problem (it gets stuck), not a **chaos** problem (errors growing over time). Time-marching fixes the training problem by giving each network a short, manageable job.

Graph 4: The first theory

With the warm start, every window length eventually breaks at T = 20.



Graph 4. Time-marching to T = 20 with the warm start, for windows of 0.25, 0.5, 1, 1.5 and 2 units. Left: error over time. Right: the error handed from each window to the next.

What you are looking at

Five runs to T = 20, each using a different window length, all with the warm start (each window's network begins as a copy of the previous one). The left plot is error over time. The right plot shows the error at each hand-off, i.e. how far off each window's end point was when it was passed on.

Before this test, a run to T = 10 had suggested a rule: at T = 10, 0.5-unit windows ran cleanly, but 1-unit windows broke at t = 8, 1.5-unit at t = 3.5 and 2-unit at t = 4.3. That looked like "the longer you go, the shorter the windows need to be". This T = 20 run was designed to test that rule.

What it shows

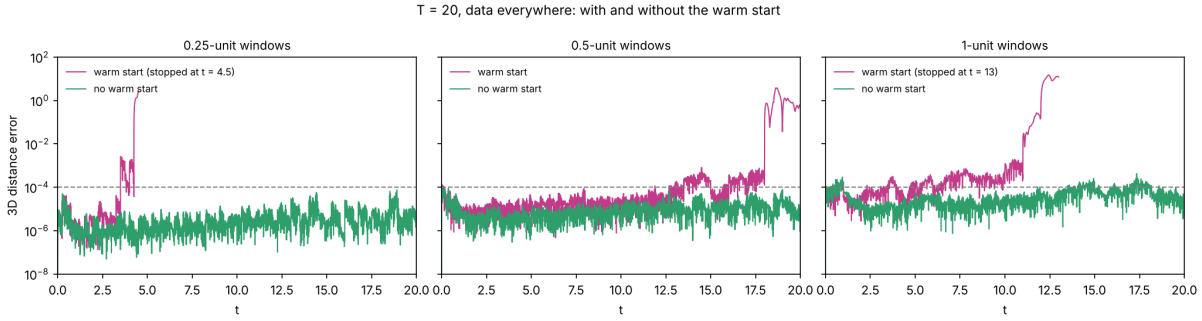
- **Every run eventually leaves the true path.** Going off the path (error above 0.1) happened at: t = 4.3 for 0.25-unit windows, t = 18.0 for 0.5, t = 11.6 for 1, t = 9.2 for 1.5, and t = 6.0 for 2.
- In each case the error is small and flat for a while, then **suddenly jumps** by a large amount. In the right plot this is one hand-off where the error leaps up. One single window's training went wrong, and every window after it started from a wrong point.
- **The rule did not hold cleanly.** 0.5 survived longest, but 0.25, the shortest window, failed earliest of all. If short windows were the answer, 0.25 should have been the best.

Key takeaway

Window length alone did not explain the failures. Something else was making individual windows get stuck. The 0.25-unit failure was the clue that led to the next graph.

Graph 5: The key finding

Remove the warm start and the failures disappear.



Graph 5. Error over time to T = 20 with the warm start (pink) and with fresh weights in every window (green), for 0.25-, 0.5- and 1-unit windows.

What you are looking at

Three panels, one for each window length. In each, the pink line is the warm-started run from Graph 4, and the green line is the same run with one change: every window's network starts from fresh random settings instead of a copy of the previous window's network.

Why we suspected the warm start

- In the failed 0.25-unit run, the training logs showed that the copied networks started *worse* than fresh ones (starting loss of 10 to 17, against about 2 to 6 for fresh networks).
- They also got slowly worse from window to window before one finally stalled completely.
- A 0.25-unit window covers less than half of one loop (a loop is about 0.62 units), so neighbouring windows look quite different. Copying a network shaped for the previous stretch is then a bad head start rather than a good one, and the damage seemed to build up with each copy.

What it shows

- **All three green runs went cleanly all the way to t = 20**, through all five wing jumps, with the error staying near or below the 10^{-4} target the whole time.
- Overall accuracy with fresh weights: 2.6×10^{-7} for 0.25-unit windows (80 windows, 224 minutes), 5.6×10^{-7} for 0.5 (40 windows, 184 minutes), and 2.1×10^{-6} for 1 unit (20 windows, 148 minutes).
- The pink lines sit above the green ones and drift upward well before they break. The warm start was quietly hurting accuracy even before it caused a failure.

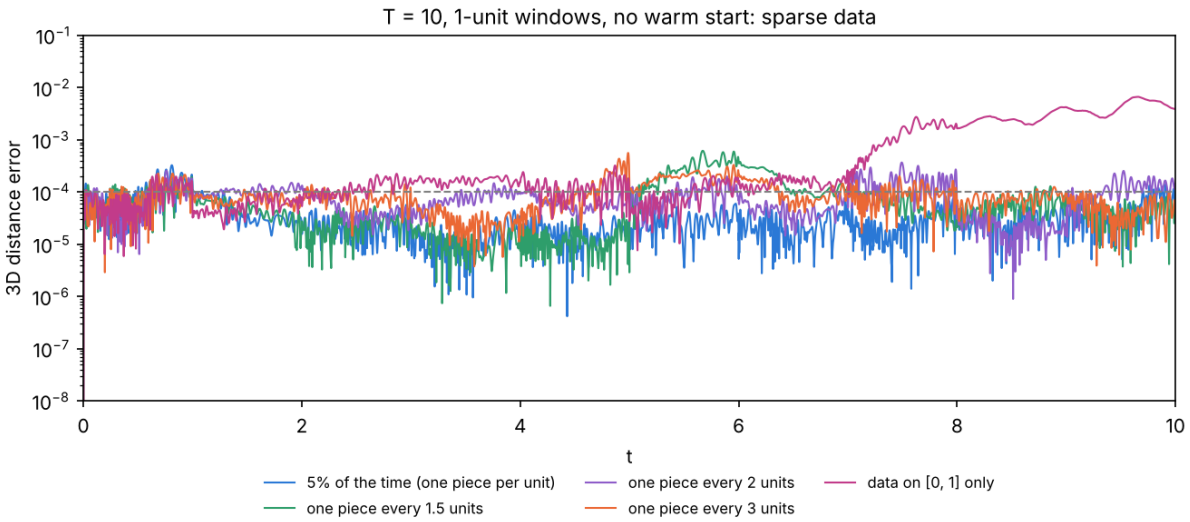
Key takeaway

Every stuck window we had seen was caused by the warm start, not by the window length, the number of windows, or the wing jumps. With fresh weights, window length only changes *how accurate* the run is (shorter is better, by about 8 times from 1 unit to 0.25), not *whether it survives*.

Why the warm start hurts is our reading of the training logs. The pattern fits a known effect in machine learning: a network that has been trained and re-trained can lose its ability to learn something new. We did not run a separate test to prove that is the mechanism.

Graph 6: How little data it needs

With fresh weights, one short data piece every 3 time units is enough.



Graph 6. Error over time to T = 10 with fresh weights and 1-unit windows, for five different amounts of data.

What you are looking at

Until now every window had data throughout. Here the networks only see a few short pieces of true data. Each piece is 0.05 time units long (about 10 measured points). The first piece is always at t = 0.5, which earlier testing showed is the one place in the first unit where data really matters (see the extra graph at the end). Each line is a different layout:

- Blue: one piece in every unit (data covers 5% of the time).
- Green: one piece every 1.5 units.
- Purple: one piece every 2 units.
- Orange: one piece every 3 units.
- Pink: true data from 0 to 1 only, and nothing at all after that.

Windows with no data in them are trained on the physics alone: the network only knows its starting point (handed over from the previous window) and the equations.

What it shows

- **Every layout reached t = 10.** The four layouts with occasional pieces all stay around 10^{-5} to 10^{-4} the whole way, with an overall held-out accuracy of about 2 to 4 × 10^{-6}. There is little difference between a piece every unit and a piece every 3 units.
- **Compare with the warm start:** in an earlier test at T = 5 with the warm start, a piece every 1.5 units gave an error of 0.002 and a piece every 2 units gave 0.30 (completely off). With fresh weights, both are about 1,000 to 100,000 times better.
- **The pink line (data on 0 to 1 only)** also reaches t = 10, but it creeps up from about t = 7 and ends around 0.004. Its error grows by about 1.7 times per unit. That steady, gradual growth is

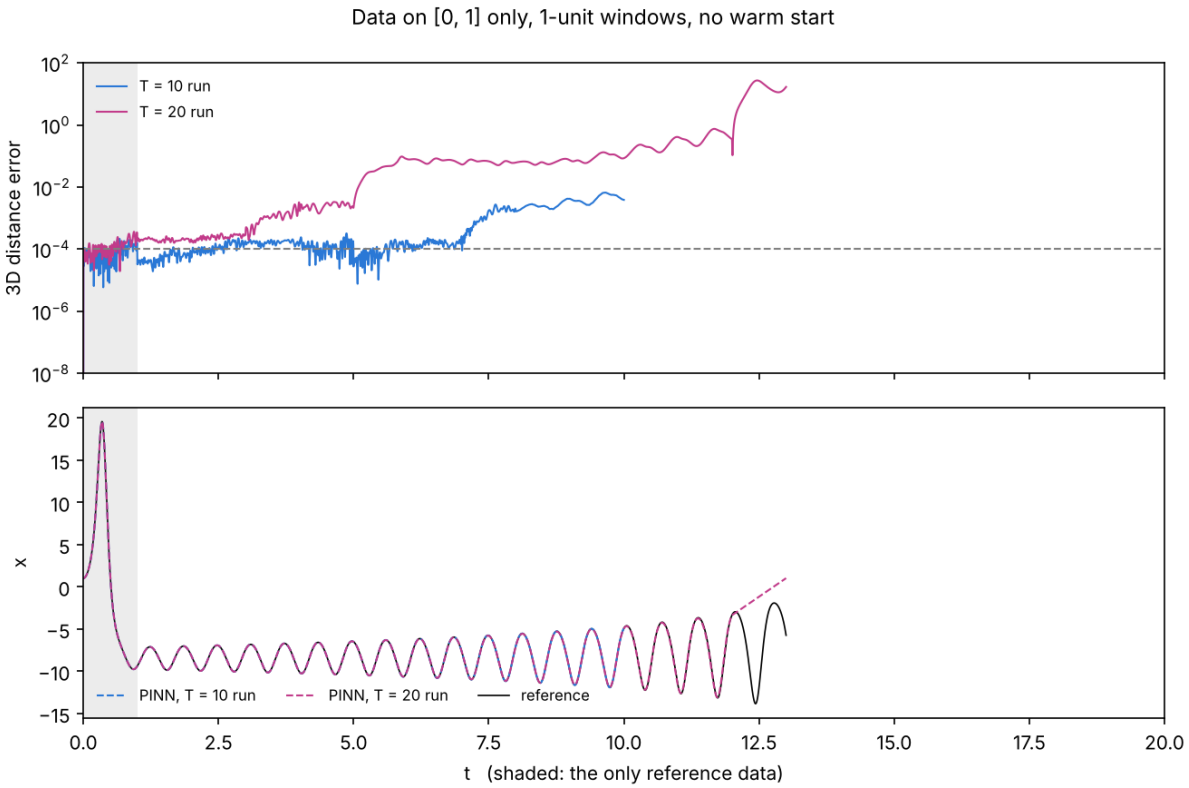
what chaos amplifying small errors looks like, rather than a window failing. That reading comes from the shape of the curve; we did not test it separately.

Key takeaway

A window does not need its own data to stay on the path. If it starts at the right point, the equations alone are enough to carry it through one time unit. The data was mostly making up for the bad starts the warm start caused. With fresh weights, a little data every few units is plenty.

Graph 7: The current limit

Data on [0, 1] only: physics alone carries the prediction to about t = 12.



Graph 7. Data on [0, 1] only (grey shading), 1-unit windows, fresh weights. Top: error over time for two runs. Bottom: x over time against the true path (black).

What you are looking at

Two runs that only ever see true data from t = 0 to 1 (the grey band). After that, every window is trained on the equations alone. Blue is the T = 10 run from Graph 6. Pink is the same set-up run to T = 20. The bottom plot shows x for each run on top of the true path.

What it shows

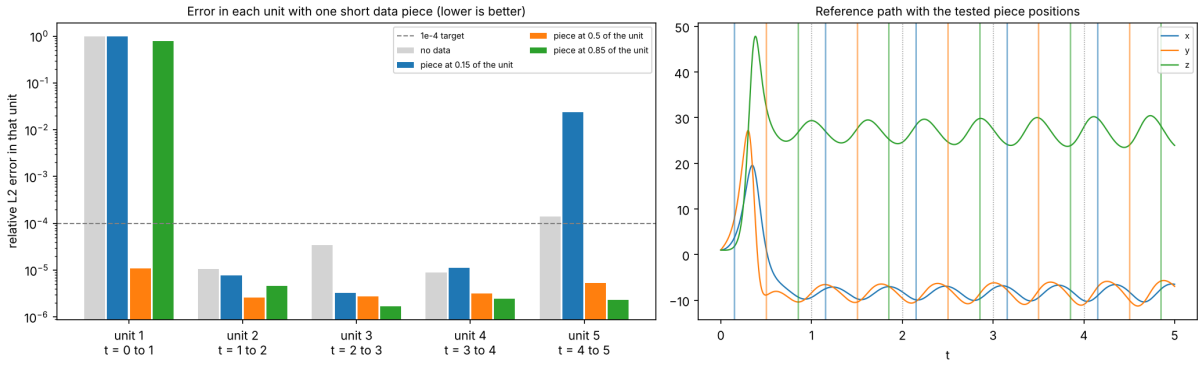
- **Both runs follow the true path to about t = 12** (bottom plot: the dashed lines sit on the black line through about 18 loops). The T = 10 run reaches its end at t = 10 with an error of about 0.004.
- **Pink run:** the window from 5 to 6 trained poorly (its loss ended near 0.0015, when the others reached about 0.000001). That added about 0.08 of error in one step, visible as the jump at t = 5. Chaos then grew it to about 0.4 by t = 11.
- **The pink run then fails in the window from 12 to 13.** In the bottom plot the prediction leaves the true path at t = 12 and heads off in a straight line in the wrong direction.
- **That window is hard, not unlucky.** In a separate repeat of this run (not drawn here), the window from 5 to 6 happened to train well and the error stayed under 0.005 to t = 12. The window from 12 to 13 was then trained three times with different random starting settings, and all three attempts got stuck, even though it started almost exactly on the true path.

- That stretch is where the path makes its deepest dip so far, with x dropping to about -14 just before the jump to the other wing at t = 13.6. With data in every window (Graph 5) the networks got through it without trouble.

Key takeaway

With data only at the start, the equations alone carry the prediction about 11 time units further, through about 18 loops. The limit is not chaos at this point: it is one hard stretch of the path that a 1-unit network cannot learn without data, the same kind of getting stuck that one network has at T = 3. Retrying with new random settings did not help, so the likely fixes are a shorter window in that stretch, or one short data piece there. Neither has been tested yet.

Extra graph: where in each unit data helps most



Graph 8. Left: error in each unit of the T = 5 path with no data, or one short data piece at 0.15, 0.5 or 0.85 of the way through the unit. Right: the true path with the tested positions marked.

What you are looking at

Each 1-unit stretch of the path from 0 to 5 was trained on its own, starting from the true point, with either no data or one short piece of data at the start, middle or end of the unit. That is 20 small runs. The left plot shows how well each one did (lower is better, log scale).

What it shows

- **Unit 1 (0 to 1) is the one that matters, and only the middle works.** With no data, or a piece at 0.15 or 0.85, it fails completely (relative error about 0.8 to 1, i.e. totally wrong). With the piece at 0.5 it is excellent (0.00001).
- The reason: the first unit contains the big early swing and the first jump to the other wing at about t = 0.5. A piece at 0.5 is the one spot that tells the network how big the swing was and which wing it lands on. A piece before the swing or well after it doesn't.
- **Units 2 to 4** are calm loops. Starting from the true point, they work even with no data. A piece of data still makes them a few times more accurate, and a piece late in the unit gives the cleanest end point to hand to the next window.
- **Unit 5** is borderline with no data and fine with a piece in the middle or at the end.

Key takeaway

This is why every sparse test in Graph 6 puts its first data piece at t = 0.5. Data is most valuable where the path does something sharp or surprising.

Putting it together

Read in order, the graphs tell one story:

1. **One network can't cover much time.** At T = 3 a single network gets stuck on a smooth wrong answer from the start (Graphs 1 to 3). This is a training failure, not chaos.
2. **Short windows fix that.** Giving each 1-unit stretch its own network makes the job easy, and accuracy improves about 100,000 times.
3. **Copying the previous network was the hidden problem.** Runs to T = 20 with the warm start kept breaking, and it first looked like a window-length rule (Graph 4). With fresh weights every window length tested ran cleanly to T = 20 (Graph 5).
4. **Very little data is needed.** With fresh weights, one short data piece every 3 units held about one-in-a-million accuracy over T = 10 (Graph 6).
5. **Physics alone goes a long way.** With data only at the very start, the equations carried the prediction to about t = 12. The limit there is one hard stretch of the path, not chaos (Graph 7).
6. **Where the data goes matters** most in the first unit, where only the middle works (Graph 8).

Key takeaway

The short version: a PINN can follow this chaotic system for a long time if each network is given a short stretch of time, starts from fresh settings, and sees a short piece of real data every few time units, placed where the path does something sharp. Short windows make training possible, fresh settings make it reliable, and occasional data guards against the rare window that can't be learned from the equations alone.

What is still open

- Each setting was run once with one random seed. Repeats would show how much of each result is luck, and the two T = 20 data-on-[0, 1] runs already show that individual windows vary.
- The window from 12 to 13 without data: trying a shorter window there, or one data piece, is the obvious next test.
- Going beyond T = 20 with data every few units, to find where chaos rather than training sets the limit.
- The explanation for why the warm start hurts comes from the training logs and fits a known effect, but was not proven with a dedicated experiment.

Where everything is

Full technical report: `results/Lorenz_PINN_Report.pdf` (Sections 16 and 17). Graph-by-graph notes: `results/horizon/graph_explanations.md`. Code: `scripts/time_marching.py` (time-marching, with --window, --no-warm-start, --stop-above and --retry-above) and `scripts/horizon_visuals.py` (all graphs).